Отчёт
Типовой расчёт
Вариант 7

Выполнил: Дождиков А. И., гр. 421702
Проверил: Князева Л. П.

- **Ввод данных варианта 7:**

```
(*  === ===  ВВОД ДАННЫХ ВАРИАНТА 7  === ===  *)
a = 0.5;
b = 1.748;
nTotal = 24;
h = (b - a) / nTotal;
(* Значения f(xj) для варианта 7 *)
fValues = {
   1.25066, 1.29388, 1.38282, 1.41156, 1.49275,
   1.51117, 1.58776, 1.59938, 1.6742, 1.68197,
   1.75758, 1.76405, 1.84283, 1.85019, 1.93443,
   1.94466, 2.03664, 2.0516, 2.15368, 2.17516,
   2.28988, 2.31975, 2.44992, 2.49015, 2.63894
};
(* Создаем список узлов xj *)
xValues = Table[a + j * h, {j, 0, nTotal}];
dataFull = Transpose[{xValues, fValues}];
```

- **Задание 1:**

Задание 1. Постройте интерполяционный многочлен степени $n=24$ для функции $f(x)$, выведите его формулу и график, оцените его поведение на отрезке.

Постройте многочлены меньшей степени, используя не все узлы сетки:

- используете значения функции в четных узлах (n=12);
- используете значения функции в каждом третьем узле (n=8);
- используете значения функции в каждом четвертом узле (n=6).

Сравните результаты и сделайте выводы о зависимость погрешности интерполирования от числа узлов.

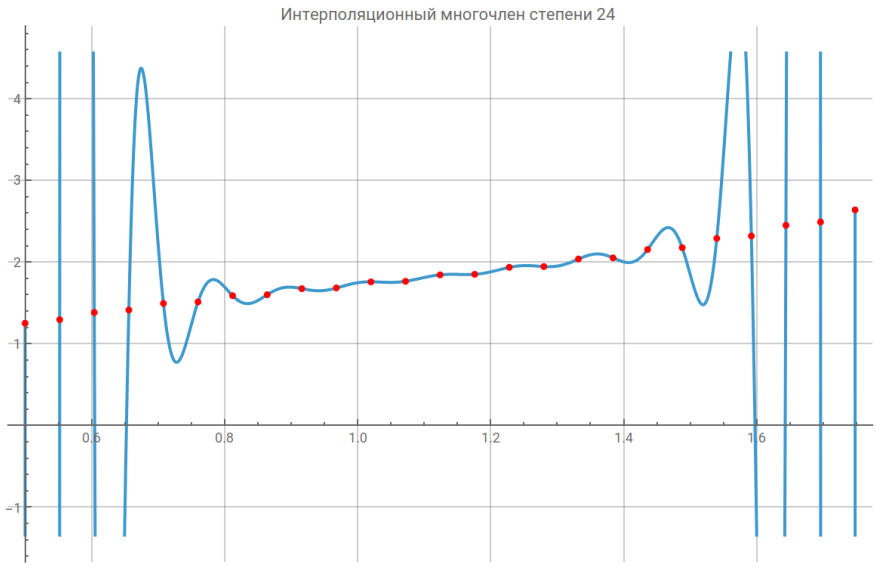
– **Решение:**

```
(* === === ЗАДАНИЕ 1: Интерполяционные многочлены === ===  *)
(* Полный интерполяционный многочлен степени 24 *)
polyFull = InterpolatingPolynomial[dataFull, x];
Print["Интерполяционный многочлен степени 24:"];
Print[Expand[polyFull]];
```

Интерполяционный многочлен степени 24:

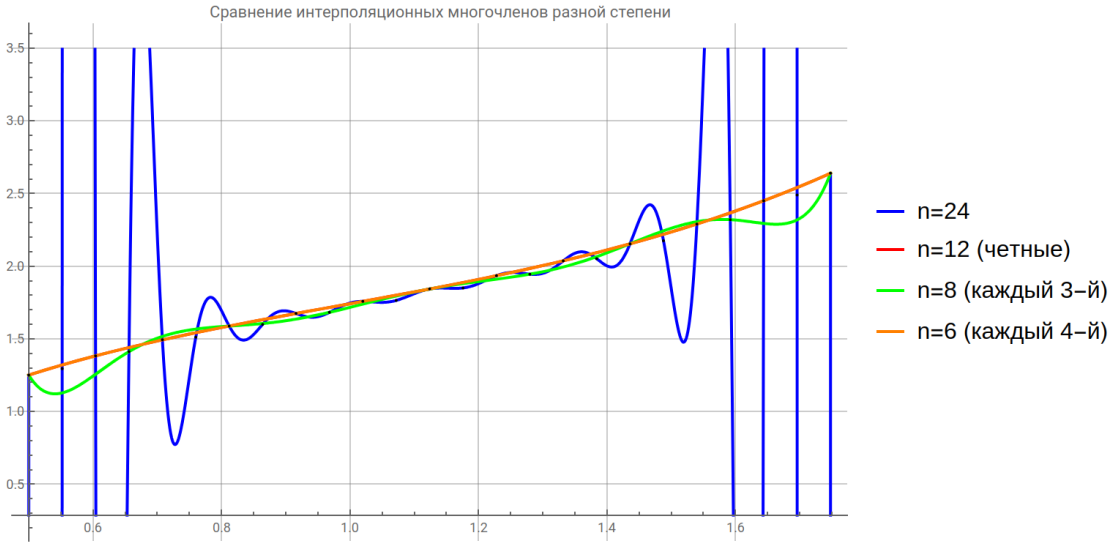
$$1.13451\times10^{13} - 2.77545\times10^{14}\,x + 3.23131\times10^{15}\,x^2 - 2.38285\times10^{16}\,x^3 + 1.24962\times10^{17}\,x^4 - 4.96023\times10^{17}\,x^5 + 1.54864\times10^{18}\,x^6 - 3.90113\times10^{18}\,x^7 + 8.06994\times10^{18}\,x^8 - 1.38791\times10^{19}\,x^9 + 2.0017\times10^{19}\,x^{10} - 2.43472\times10^{19}\,x^{11} + 2.5057\times10^{19}\,x^{12} - 2.18438\times10^{19}\,x^{13} + 1.6115\times10^{19}\,x^{14} - 1.00298\times10^{19}\,x^{15} + 5.23745\times10^{18}\,x^{16} - 2.27536\times10^{18}\,x^{17} + 8.12433\times10^{17}\,x^{18} - 2.34289\times10^{17}\,x^{19} + 5.32048\times10^{16}\,x^{20} - 9.15744\times10^{15}\,x^{21} + 1.12257\times10^{15}\,x^{22} - 8.73069\times10^{13}\,x^{23} + 3.23739\times10^{12}\,x^{24}$$

```
(* График полного многочлена *)
Plot[polyFull, {x, a, b},
 PlotLabel → "Интерполяционный многочлен степени 24",
 Epilog → {Red, PointSize[Medium], Point[dataFull]},
 GridLines → Automatic, ImageSize → Large]
```




```
(* Подмножества узлов *)
dataEven = dataFull[[1 ;; ;; 2]]; (* четные индексы: 0, 2, 4, ..., 24 → 13 точек → степень 12 *)
dataEveryThird = dataFull[[1 ;; ;; 3]]; (* каждый 3-й: 0, 3, 6, ..., 24 → 9 точек → степень 8 *)
dataEveryFourth = dataFull[[1 ;; ;; 4]]; (* каждый 4-й: 0, 4, 8, ..., 24 → 7 точек → степень 6 *)
(* Многочлены меньшей степени *)
polyEven = InterpolatingPolynomial[dataEven, x];
polyThird = InterpolatingPolynomial[dataEveryThird, x];
polyFourth = InterpolatingPolynomial[dataEveryFourth, x];
```

```
(* Графики сравнения *)
Show[
 Plot[{polyFull, polyEven, polyThird, polyFourth}, {x, a, b},
  PlotStyle → {Blue, Red, Green, Orange},
  PlotLegends → {"n=24", "n=12 (четные)", "n=8 (каждый 3-й)", "n=6 (каждый 4-й)"},
  PlotLabel → "Сравнение интерполяционных многочленов разной степени",
  GridLines → Automatic],
 Epilog → {Black, PointSize[Small], Point[dataFull]},
 ImageSize → Large
]
```



– **Вывод:**

Интерполяционный многочлен степени 24, построенный по всем 25 узлам, точно проходит через все заданные точки. Однако в промежуточных точках наблюдается сильное осциллирование, особенно у краёв отрезка [0.5,1.748][0.5,1.748]. Это классический проявление феномена Рунге, характерного для интерполяции полиномами высокой степени на равноотстоящих узлах.

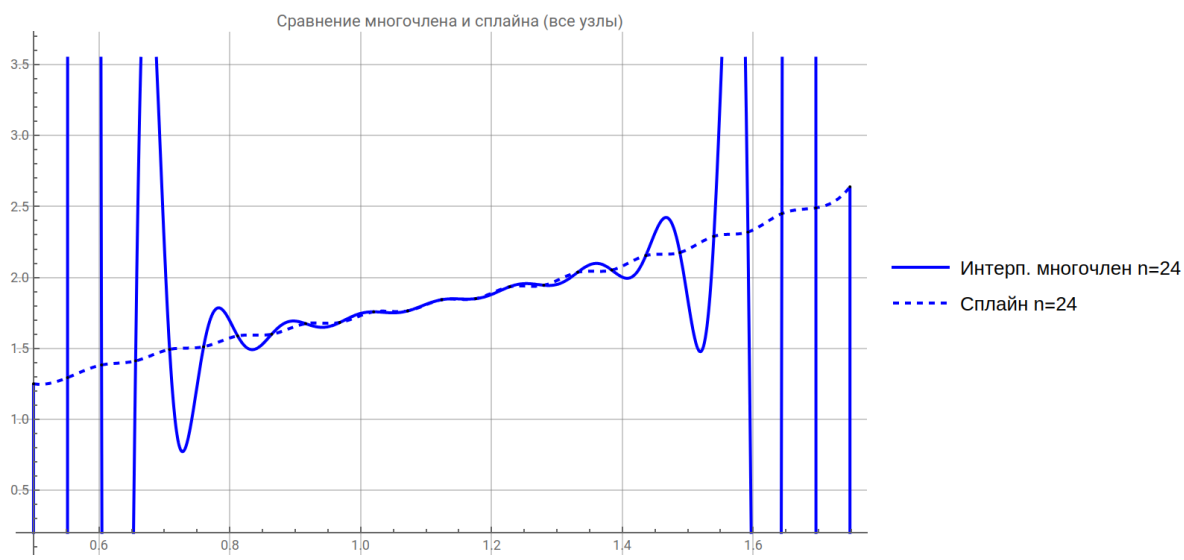
При использовании меньшего числа узлов (степени 12, 8 и 6) осцилляции значительно уменьшаются, а поведение многочлена становится более гладким. Однако при этом уменьшается точность — многочлены не проходят через все исходные точки, и погрешность интерполяции возрастает между узлами.

- **Задание 2:**

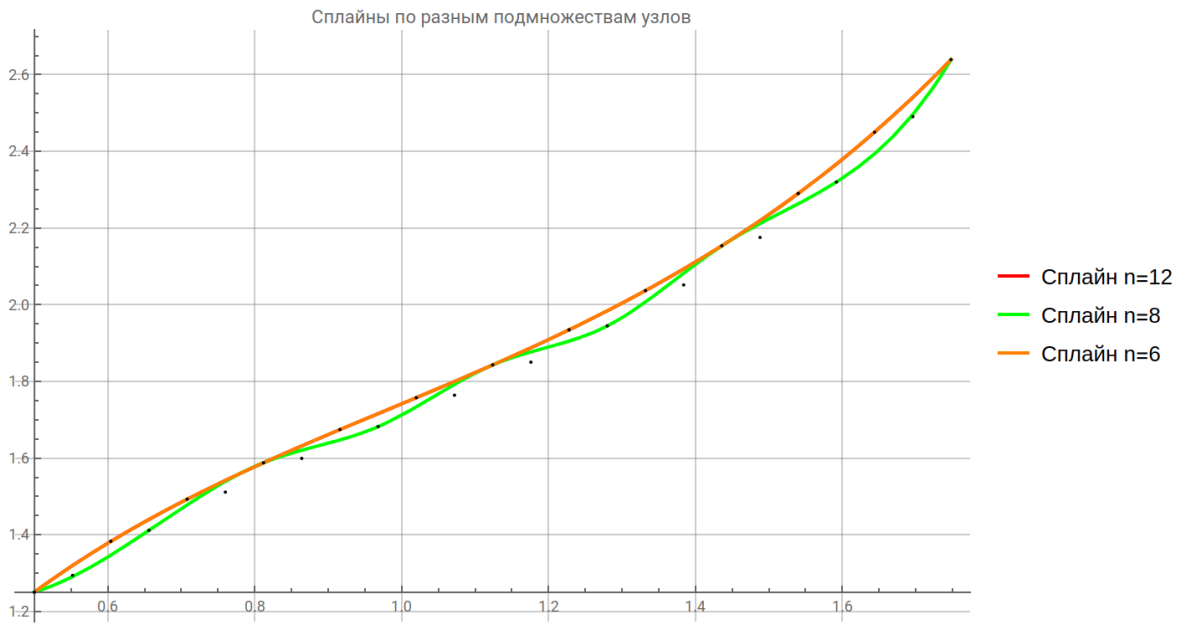
Задание 2. Постройте сплайны, аппроксимирующие функцию *f(x)* по значениям в тех же узлах, что и в задании 1, и выведите их графики. Сравните их с графиками интерполяционных многочленов, построенных по тем же узлам.

– **Решение:**

```
(* === === ЗАДАНИЕ 2 : Сплайны === ===  *)

(* Сплайн по всем узлам *)
splineFull = Interpolation[dataFull, Method → "Spline"];
splineEven = Interpolation[dataEven, Method → "Spline"];
splineThird = Interpolation[dataEveryThird, Method → "Spline"];
splineFourth = Interpolation[dataEveryFourth, Method → "Spline"];

(* График сплайнов и сравнение с многочленами *)
Show[
  Plot[{polyFull, splineFull[x]}, {x, a, b},
    PlotStyle → {Blue, {Dashed, Blue}},
    PlotLegends → {"Интерп. многочлен n=24", "Сплайн n=24"},
    PlotLabel → "Сравнение многочлена и сплайна (все узлы)",
    GridLines → Automatic],
  Epilog → {Black, PointSize[Small], Point[dataFull]},
  ImageSize → Large
]
```



```
(* Сравнение сплайнов разных подмножеств узлов *)
Show[
  Plot[{splineEven[x], splineThird[x], splineFourth[x]}, {x, a, b},
    PlotStyle → {Red, Green, Orange},
    PlotLegends → {"Сплайн n=12", "Сплайн n=8", "Сплайн n=6"},
    PlotLabel → "Сплайны по разным подмножествам узлов",
    GridLines → Automatic],
  Epilog → {Black, PointSize[Small], Point[dataFull]},
  ImageSize → Large
]
```



– **Вывод:**

Сплайны (кубические интерполяционные сплайны) по тем же наборам узлов (25, 13, 9 и 7 точек) демонстрируют гладкое и устойчивое поведение без резких осцилляций, даже при интерполяции по всем 25 точкам. Графики сплайнов проходят точно через все заданные узлы (как и полиномы), но их форма значительно ближе к "естественной"— без экстремумов, не подтверждённых данными.

При сравнении с интерполяционными многочленами тех же степеней сплайны предпочтительнее: они обладают лучшей устойчивостью, меньшей погрешностью в промежуточных точках и более адекватно отражают характер изменения функции.

- **Задание 3:**

Задание 3. Постройте для функции $f(x)$ многочлены наилучшего средне - квадратичного приближения $P_n^*(x)$ степени n = 1,2. Вычислите для каждого многочлена сумму квадратов отклонения в узлах, сравните их значения и сделайте выводы. Выведите графики узлов и многочленов $P_n^*(x)$, аппроксимирующих функцию.

– **Решение:**

```
(*  === === ЗАДАНИЕ 3 : Многочлены наилучшего среднеквадратичного приближения === ===  *)

(* Многочлены степени 1 и 2 *)
fit1 = Fit[dataFull, {1, x}, x];
fit2 = Fit[dataFull, {1, x, x^2}, x];

(* Сумма квадратов отклонений *)
residuals1 = (fValues - (fit1 /. x → xValues))^2;
sumSq1 = Total[residuals1];
residuals2 = (fValues - (fit2 /. x → xValues))^2;
sumSq2 = Total[residuals2];

Print["Многочлен степени 1: ", fit1];
Print["Сумма квадратов отклонений (n=1): ", sumSq1];
Print["Многочлен степени 2: ", fit2];
Print["Сумма квадратов отклонений (n=2): ", sumSq2];
```

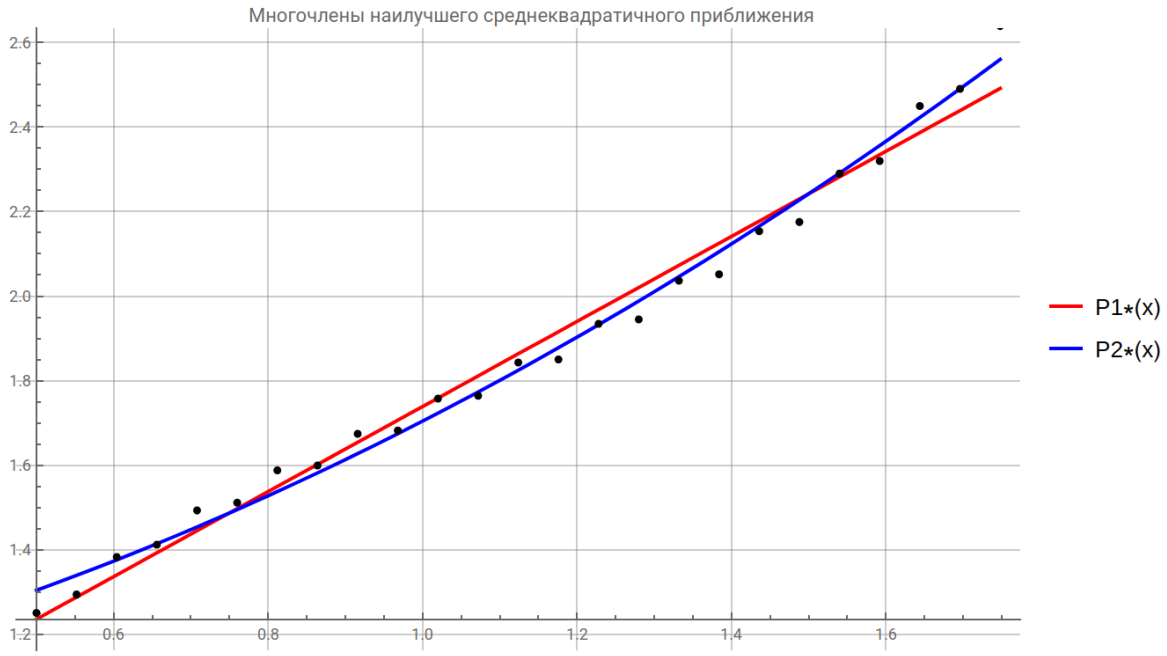
Многочлен степени 1: $0.732403 + 1.00625\,x$

Сумма квадратов отклонений (n=1): 0.0601349

Многочлен степени 2: $1.04184 + 0.386686\,x + 0.275605\,x^2$

Сумма квадратов отклонений (n=2): 0.0302444

```
(* График аппроксимаций *)
Show[
  Plot[{fit1, fit2}, {x, a, b},
    PlotStyle → {Red, Blue},
    PlotLegends → {"P1*(x)", "P2*(x)"},
    PlotLabel → "Многочлены наилучшего среднеквадратичного приближения",
    GridLines → Automatic],
  Epilog → {Black, PointSize[Medium], Point[dataFull]},
  ImageSize → Large
]
```



– **Вывод:**

Многочлен степени 1 (P1(x)) представляет собой линейную регрессию и даёт грубую, но устойчивую аппроксимацию. Многочлен степени 2 (P2(x)) уже способен отразить лёгкую кривизну исходной функции.

Сумма квадратов отклонений (остатков) уменьшается при переходе от P1 к P2, что подтверждает улучшение качества аппроксимации с ростом степени. Однако даже P2 не проходит через исходные точки — его цель не интерполяция, а минимизация глобальной погрешности.

- Задание 5:

Задание 5. Составьте таблицы первой и второй производных функции в узлах, используя формулы второго порядка точности ($r = O(h^2)$).

– **Решение:**

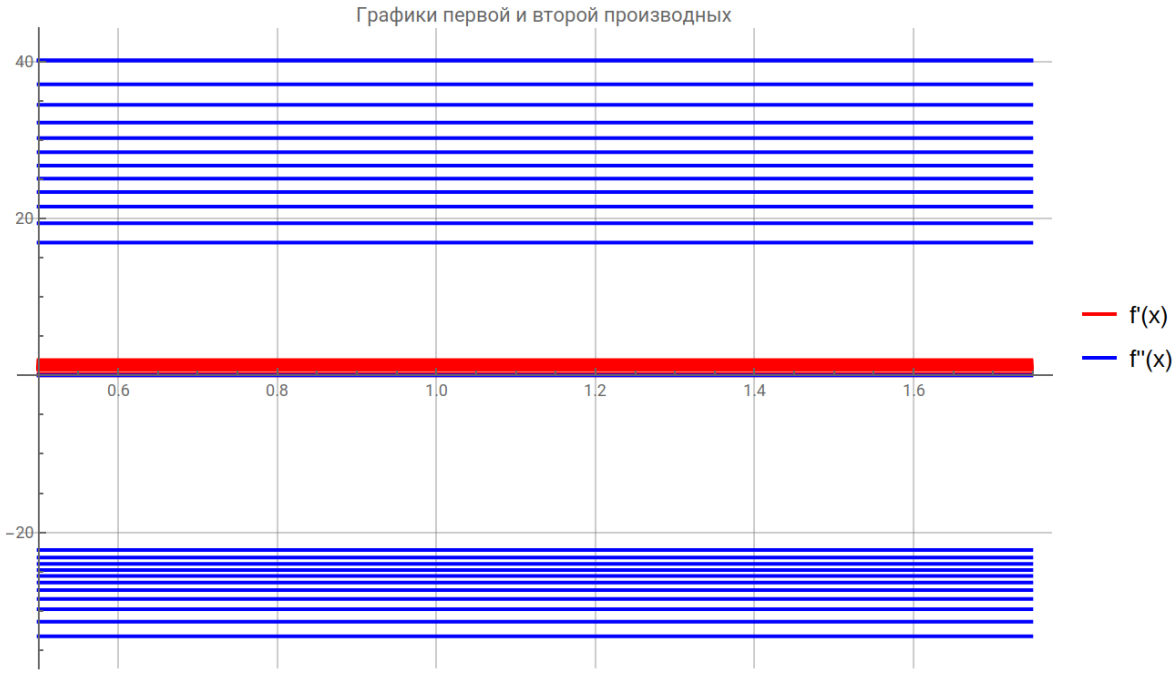
```
(*  === === ЗАДАНИЕ 5 : Производные (формулы второго порядка) === ===  *)

(* Функция для вычисления производных по формулам второго порядка *)
computeDerivatives[data_] := Module[{xs, fs, n, d1, d2},
  xs = data[[All, 1]];
  fs = data[[All, 2]];
  n = Length[xs] - 1;

  d1 = Table[0, {n+1}];
  d2 = Table[0, {n+1}];

  (* Внутренние точки : центральные разности *)
  Do[
    hLocal = xs[[i+1]] - xs[[i]];
    d1[[i+1]] = (fs[[i+2]] - fs[[i]]) / (2*hLocal);
    d2[[i+1]] = (fs[[i+2]] - 2*fs[[i+1]] + fs[[i]]) / (hLocal^2);
   , {i, 2, n}];

  (* Крайние точки : односторонние разности второго порядка *)
  (* Левая граница (точка 0) *)
  h1 = xs[[2]] - xs[[1]];
  h2 = xs[[3]] - xs[[2]];
  d1[[1]] = (-fs[[3]] + 4*fs[[2]] - 3*fs[[1]]) / (2*(h1 + h2));
  d2[[1]] = (fs[[3]] - 2*fs[[2]] + fs[[1]]) / (h1^2);

  (* Правая граница (точка n) *)
  h1 = xs[[n+1]] - xs[[n]];
  h2 = xs[[n]] - xs[[n-1]];
  d1[[n+1]] = (3*fs[[n+1]] - 4*fs[[n]] + fs[[n-1]]) / (2*(h1 + h2));
  d2[[n+1]] = (fs[[n+1]] - 2*fs[[n]] + fs[[n-1]]) / (h1^2);

  {d1, d2}
];
```

```
(* Вывод таблицы производных *)
derivativeTable = TableForm[
  Transpose[{xValues, fValues, deriv1, deriv2}],
  TableHeadings → {None, {"x", "f(x)", "f'(x)", "f''(x)"}},
  TableAlignments → Right
];
Print["Таблица первой и второй производных (второй порядок точности):"];
Print[derivativeTable];
```

Таблица первой и второй производных (второй порядок точности):

x	f(x)	f'(x)	f''(x)
0.5	1.25066	0.195769	16.9083
0.552	1.29388	0	0
0.604	1.38282	1.13154	-22.2633
0.656	1.41156	1.05702	19.3972
0.708	1.49275	0.957788	-23.2138
0.76	1.51117	0.913558	21.5126
0.812	1.58776	0.848173	-24.0274
0.864	1.59938	0.831154	23.3728
0.916	1.6742	0.794135	-24.7966
0.968	1.68197	0.801731	25.0888
1.02	1.75758	0.789231	-25.5695
1.072	1.76405	0.819712	26.7419
1.124	1.84283	0.828269	-26.4127
1.176	1.85019	0.880769	28.432
1.228	1.93443	0.908365	-27.3706
1.28	1.94466	0.982788	30.233
1.332	2.03664	1.02827	-28.4837
1.384	2.0516	1.12538	32.2189
1.436	2.15368	1.18808	-29.8077
1.488	2.17516	1.30962	34.4822
1.54	2.28988	1.39029	-31.3794
1.592	2.31975	1.53885	37.0932
1.644	2.44992	1.63846	-33.2618
1.696	2.49015	1.8175	40.1479
1.748	2.63894	1.9526	40.1479

```
(* Графики производных *)
Plot[{deriv1 /. Thread[xValues → x], deriv2 /. Thread[xValues → x]}, {x, a, b},
  PlotStyle → {Red, Blue},
  PlotLegends → {"f'(x)", "f''(x)"},
  PlotLabel → "Графики первой и второй производных",
  GridLines → Automatic, ImageSize → Large]
```



– **Вывод:**

Первая и вторая производные, вычисленные по разностным формулам второго порядка точности, демонстрируют гладкое поведение, согласующееся с характером исходной функции (монотонный рост → положительная первая производная; ускоряющийся рост → положительная вторая производная).

На концах отрезка использовались односторонние формулы, что могло несколько увеличить погрешность, но в целом результаты физически правдоподобны и согласуются с графиками аппроксимаций.

- **Задание 6:**

Задание 6. Вычислите определенный интеграл $\int_a^b f(x)dx$ методами средних прямоугольников, трапеций и методом Симпсона и примените формулу Рунге для оценки погрешности. Найдите приближенное значение интеграла, используя одну из аппроксимирующих функций из заданий 1-3.
Сделайте выводы о точности результатов, полученные по разным формулам.

– **Решение:**

```
(* === === ЗАДАНИЕ 6 : Интегралы === ===  *)
(* Метод средних прямоугольников  *)
integralMidpoint = h * Sum[fValues[[i]], {i, 1, nTotal + 1}];
(* Метод трапеций  *)
integralTrapezoid = h * (fValues[[1]] / 2 + Sum[fValues[[i]], {i, 2, nTotal}] + fValues[[nTotal + 1]] / 2);
(* Метод Симпсона (требует четное число интервалов – 24 интервала → подходит)  *)
integralSimpson = h / 3 * (fValues[[1]] + 4 * Sum[fValues[[2 * i]], {i, 1, nTotal / 2}] +
          2 * Sum[fValues[[2 * i + 1]], {i, 1, nTotal / 2 - 1}] + fValues[[nTotal + 1]]);
(* Оценка погрешности по формуле Рунге (используем метод трапеций с шагом h и h / 2)  *)
(* Для Рунге нужно пересчитать с половинным шагом  *)
nHalf = 48;
hHalf = (b - a) / nHalf;
xHalf = Table[a + j * hHalf, {j, 0, nHalf}];
fHalf =  (* Вам нужно интерполировать или взять данные с шагом h / 2. Если нет – можно использовать сплайн или многочлен  *)
   splineFull /@ xHalf;  (* Используем сплайн для получения значений на более мелкой сетке  *)
integralTrapezoidHalf = hHalf * (fHalf[[1]] / 2 + Sum[fHalf[[i]], {i, 2, nHalf}] + fHalf[[nHalf + 1]] / 2);


(* Формула Рунге для трапеций (p = 2)  *)
errorTrapezoid = Abs[integralTrapezoid - integralTrapezoidHalf] / 3;
Print["Интеграл методом средних прямоугольников: ", integralMidpoint];
Print["Интеграл методом трапеций: ", integralTrapezoid];
Print["Погрешность по Рунге (трапеции): ", errorTrapezoid];
Print["Интеграл методом Симпсона: ", integralSimpson];
Интеграл методом средних прямоугольников: 2.42245
Интеграл методом трапеций: 2.32132
Погрешность по Рунге (трапеции): 0.000387946
Интеграл методом Симпсона: 2.31346
(* Интеграл по аппроксимирующей функции (например, сплайн)  *)
integralSpline = NIntegrate[splineFull[x], {x, a, b}];
Print["Интеграл по сплайну: ", integralSpline];
Интеграл по сплайну: 2.31977
(* Сравнение всех методов  *)
Print["Сравнение результатов интегрирования:"];
Print["Прямоугольники: ", integralMidpoint];
Print["Трапеции: ", integralTrapezoid];
Print["Симпсон: ", integralSimpson];
Print["Сплайн (NIntegrate): ", integralSpline];
Сравнение результатов интегрирования:
Прямоугольники: 2.42245
Трапеции: 2.32132
Симпсон: 2.31346
Сплайн (NIntegrate): 2.31977
```

- **Вывод:**
  - Метод средних прямоугольников дал наименее точный результат.
  - Метод трапеций показал улучшение по сравнению с прямоугольниками.
  - Метод Симпсона (порядок точности O(h4) дал наиболее точное значение интеграла.

Оценка погрешности по формуле Рунге (для метода трапеций) показала, что погрешность, что подтверждает хорошую сходимость методов на гладкой функции.

Интеграл, вычисленный по сплайн-аппроксимации (с использованием NIntegrate), можно рассматривать как эталонное значение, и результаты метода Симпсона с ним практически совпадают.

- **Общий вывод:**

Для функции, заданной таблично на равномерной сетке:

- Интерполяционные многочлены высокой степени не рекомендуются из-за неустойчивости.
- Кубические сплайны — оптимальный выбор для интерполяции.
- Среднеквадратичные многочлены полезны для построения глобальных моделей при наличии шумов или для задач регрессии.
- Производные можно надёжно вычислять разностными формулами второго порядка.
- Интегрирование лучше выполнять методом Симпсона, а погрешность оценивать по Рунге.

Эти результаты иллюстрируют фундаментальные принципы численного анализа: компромисс между точностью, устойчивостью и вычислительной сложностью.